Abliteration: How We Removed a Behaviour from a 128B Model Using Linear Algebra

How We Removed a Behaviour from a 128-Billion Parameter AI Model Using Linear Algebra

A walkthrough of abliteration: what it is, why it works, and how we did it.

There is a popular misconception that removing an unwanted behaviour from a large language model (LLM) requires retraining it — feeding it thousands of new examples, running gradient descent for days, and hoping the change sticks without breaking something else. For certain changes, that is true. But for a specific class of learned behaviours, there is a more surgical option. We recently applied one to Mistral Medium 3.5, a 128-billion parameter model, and the result was a permanently modified model that fits in the same file, runs in the same tools, and required no retraining at all.

The technique is called **abliteration**. It is grounded in a surprising finding from interpretability research, it involves a surprisingly small amount of linear algebra, and it is worth understanding properly — because what it reveals about how these models work is almost as interesting as what it does.

Before going further, it is worth being clear about what we mean by "removing a behaviour." We are not talking about jailbreaking — exploiting a loophole in the model's existing rules that a future update could patch. We are not talking about prompting the model differently. We are making a permanent, mathematical change to the model's weights so that the internal signal responsible for refusal behaviour is structurally absent. The model is not being tricked; it is being modified. That distinction matters both technically and ethically, and we will return to the ethical side at the end.

The surprising finding: behaviours live in directions

Before we get to the technique itself, we need the insight that makes it possible.

Inside a transformer model, every token — every word fragment being processed — is represented as a vector: a long ordered list of numbers. In the model we worked with, that list has 12,288 numbers. This vector is called the **residual stream**, and it flows through all 88 layers of the model. At each layer, the model reads the current 12,288 numbers, passes them through several weight matrices, and **adds** the result back onto the residual stream. Layers never erase the stream — they only annotate it. By the final layer, the accumulated values are what the model uses to decide what to say next.

So what does "passes them through several weight matrices" actually involve? Each layer contains **two sub-blocks in sequence: an attention block and an MLP block.** Each sub-block is made of weight matrices, and each matrix works the same fundamental way — it takes the incoming 12,288 numbers and produces a new set of numbers, where **every output number is a weighted sum of all 12,288 inputs.** That "weighted sum" is the dot product: multiply each input by its corresponding weight, add the results together, and you have one output number. Repeat with a different set of weights for each output number you need.

The attention block first uses three matrices — `q_proj`, `k_proj`, and `v_proj` — to turn each token's vector into three specialised vectors: a *query* ("what am I looking for?"), a *key* ("what do I offer?"), and a *value* ("what will I contribute if selected?"). The model compares queries against keys across every token in the sequence to decide which tokens should pay attention to which — this is the step where a word like "it" can reach back and pull in information from the noun it refers to. The gathered result is then passed through a fourth matrix, `o_proj`, which produces the attention block's contribution to add back onto the residual stream.

The MLP block is where most of the raw computation happens. It first uses two matrices — `up_proj` and `gate_proj` — to expand the 12,288 numbers into a much wider 49,152-number space (four times larger), applies a nonlinear function that lets the model represent more complex relationships, and then uses a third matrix, `down_proj`, to compress the result back down to 12,288 numbers — its contribution to add onto the residual stream.

So "passing through several weight matrices" is really seven matrices per layer, each performing that same weighted-sum operation, arranged into an attention block that mixes information *between tokens* and an MLP block that does heavier processing *within each token*. Multiply that by 88 layers and you have the full forward pass. Keep an eye on two of those seven matrices — `o_proj` and `down_proj` — because they turn out to be the only ones whose output is written onto the residual stream, and that makes them the targets for everything that follows.

It is worth pausing on what 12,288 numbers actually represents. You might be tempted to imagine each number as a labelled feature: "number 1 = politeness," "number 2 = danger," and so on. That is not how it works. The 12,288 numbers are not human-readable features; they are abstract coordinates in a high-dimensional mathematical space. Meaning is not stored in individual coordinates — it is encoded in *patterns across many coordinates at once*. A concept like "this is a harmful request" does not live in one slot. It lives as a particular shape: a particular angle, or *direction*, through the space.

This is a subtle but important shift in how to think about the model. The useful unit of meaning is not a single number but a **direction** — a vector pointing a particular way through the 12,288-dimensional space. A direction is defined by giving all 12,288 coordinates a value simultaneously: "0.3 of dimension 1, minus 0.7 of dimension 2, 0.1 of dimension 3…" and so on. Think of it as a recipe specifying a particular orientation in a very large space.

Now, you might expect that a complex behaviour like "refusing a harmful request" would be smeared across this space in some hopelessly entangled way — a different, incomprehensible combination for every prompt, impossible to isolate. The 2023 Arditi et al. paper found otherwise. It showed that, at least for refusal, the model's internal "am I about to refuse?" state is largely controlled by **a single consistent direction** through the 12,288-dimensional space. The same direction, recognisably present, across a wide variety of different harmful prompts.

The research finding is this: push a token's internal state *toward* that direction and the model refuses more, even on harmless requests. Push it *away* and refusals decrease. Remove that component entirely, and the model largely stops refusing. This is called the **linear representation hypothesis** — the idea that many high-level model behaviours compress down to a single steerable direction.

This was not guaranteed to be true. The reasonable prior was that alignment training — the process of teaching the model to refuse — would distribute refusal diffusely across billions of weights in a way that resisted surgical removal. The Arditi finding is genuinely surprising. It suggests that despite the complexity of the training process, certain high-level behaviours end up organised linearly in activation space. That organisation is what abliteration exploits.

If that is true, it hands you a lever.

Finding the direction: two clouds of dots

The refusal direction does not come pre-labelled inside the model. It has to be *calculated* from evidence.

The method is called **difference of means**, and it is almost elegant in its simplicity. You take two sets of prompts: 400 harmful or refusal-triggering ones, and 400 harmless ones. You run both sets through the model, but instead of reading the output, you eavesdrop on the model's internal state midway through — specifically, the 12,288-number vector at target layers for the last token of each prompt.

No weights change during this step. It is a purely observational pass: run the model as normal, intercept the activation vectors at the layers of interest, write them down, move on. The single question being asked is: *on average, how does the model's internal state differ when processing a harmful prompt versus a harmless one?*

Exactly where the vector comes from. This is the step that is easy to gloss over, so let us be precise. You feed one prompt — say "How do I make a bomb?" — into the model and let it run a normal forward pass. As it runs, every token in the prompt has its own 12,288-number residual stream vector at every one of the 88 layers. That is a lot of vectors: `number_of_tokens × 88 layers`, each 12,288 numbers wide. You do not keep all of them. For each prompt you capture **one specific vector**: the residual stream at a chosen target layer (say layer 40), read off at the **last token position** of the prompt.

Why the last token? Because in a decoder-only transformer, the last token is the only position that has "seen" the entire prompt through attention. Its residual stream vector is the model's most complete summary of the whole input — and it is the vector the model is about to use to decide what to generate next. If the model is going to refuse, the intention to refuse is most cleanly present right there. So the recipe per prompt is: run

forward, reach into layer 40, grab the 12,288 numbers sitting at the final token, save them as that prompt's activation vector. In practice this is done with a *forward hook* — a small callback registered on the layer that copies out the tensor as it passes through.

Do this for all 400 harmful prompts and you have 400 vectors, each 12,288 numbers long. Do it for all 400 harmless prompts and you have 400 more. These 800 vectors are the raw data for everything that follows.

Imagine plotting all of these activation vectors as dots in a very high-dimensional space (squashed to 2D to draw it). The 400 harmful-prompt activations cluster in one region — call it the red cloud. The 400 harmless-prompt activations cluster in another — the green cloud. The model really does end up in measurably different internal territory depending on what kind of prompt it received. The refusal direction is simply the arrow from the centre of the green cloud to the centre of the red cloud.

How you average a pile of vectors. "The mean of the red cloud" means the average vector, and averaging vectors is done **coordinate by coordinate**. You add up all 400 harmful vectors position-by-position — all the first coordinates summed, all the second coordinates summed, and so on across all 12,288 positions — then divide each sum by 400. The result is a single 12,288-number vector whose first coordinate is the average of all 400 first coordinates, and so on. It is the "centre of mass" of the red cloud.

```
mean_harmful[i] = (v₁[i] + v₂[i] + ... + v₄₀₀[i]) / 400     for each
coordinate i = 1 ... 12,288
```

In code this is a single operation on a stack of vectors — you place the 400 vectors into a `400 × 12,288` array and take the mean down the first axis, collapsing it to one `12,288` vector:

```
import numpy as np

harmful  = np.stack(harmful_vectors)    # shape (400, 12288)
harmless = np.stack(harmless_vectors)   # shape (400, 12288)

mean_harmful  = harmful.mean(axis=0)    # shape (12288,)
mean_harmless = harmless.mean(axis=0)   # shape (12288,)
```

Now that you have the two centre-of-mass vectors, the refusal direction is just their difference — subtracted, again, coordinate by coordinate:

```
r_raw = mean_harmful − mean_harmless
```

The resulting vector points *from* the typical harmless state *toward* the typical refusal state. It is the arrow that best summarises the systematic difference between the two clouds. Finally, normalise it to length 1 by dividing every coordinate by the vector's Euclidean length — the square root of the sum of squared coordinates — to get a pure direction vector r̂ (pronounced "r-hat") with no magnitude attached, just direction:

```
||r_raw|| = √(r₁² + r₂² + ... + r₁₂₂₈₈²)
r̂ = r_raw / ||r_raw||
```

Normalising matters because we want r̂ to measure *pure direction*. If we used the raw unnormalised vector, the dot product `r_raw · a` would vary depending on how far apart the two clouds happened to be, not just on the direction of separation. With a unit vector,

the dot product cleanly answers: “how far does this activation extend in the refusal direction?”

Why the prompt pairs need to be matched. The subtraction only isolates what is *systematically different* between the two piles. If the harmful pile happened to be all chemistry questions and the harmless pile all cooking questions, the computed direction would capture “chemistry vs. cooking” on top of “refusal vs. harmless” — you would be measuring the wrong thing. The best practice is near-minimal pairs: same template, same approximate topic, only the harmful intent changed. “How do I make a bomb?” paired against “How do I make a cake?” lets the subtraction cancel topic, grammar, and register, leaving only the refusal signal. This failure mode — computing a direction contaminated by irrelevant features — is called *direction leakage*, and it is the main thing that makes a poorly computed $\hat{r}$ ineffective.

Which layers to eavesdrop on. The refusal signal does not appear equally at every layer. Early layers (roughly 1–20) are mostly processing low-level syntax and token identity — the two clouds are barely separated there. Separation grows in the middle-to-late layers (roughly 20–60) as the model consolidates higher-level semantic meaning. Heretic targets the full middle range and aggregates across layers to produce a robust direction estimate.

A separate held-out set of 100+100 prompts is kept to measure the result afterward. Before any editing, the model refused 89 of 100 harmful test prompts. After editing: 6 out of 100.

Removing the direction: one formula

Once you have $\hat{r}$, removing refusal from any activation vector a is a single operation called **vector projection**:

$$a_{clean} = a - (\hat{r} \cdot a) \times \hat{r}$$

Break it down step by step:

- $\hat{r} \cdot a$ is the **dot product** — it multiplies matching coordinates and sums the results: $\hat{r}_1 \times a_1 + \hat{r}_2 \times a_2 + \ldots + \hat{r}_{12288} \times a_{12288}$. The result is a single number representing “how much of a points in the refusal direction.” A large positive number means the activation is strongly refusal-flavoured. Near zero means it is roughly perpendicular — no particular lean toward or away from refusal.

- Multiplying that scalar back by $\hat{r}$ gives back the actual refusal *component* as a vector — the piece of a that lies along the refusal direction. This is the “refusal shadow” cast by a.

- Subtracting that component removes exactly that piece and leaves everything else in a perfectly untouched.

In a toy 2D example where $\hat{r} = [0, 1]$ (refusal is straight up the y-axis) and $a = [0.9, 0.8]$: the dot product is $0 \times 0.9 + 1 \times 0.8 = 0.8$, the refusal component is $0.8 \times [0, 1] = [0, 0.8]$, and subtracting gives $[0.9, 0.8] - [0, 0.8] = [0.9, 0.0]$. The model’s topic knowledge (x-axis) is intact; the refusal lean (y-axis) is zeroed out.

In a real model, $\hat{r}$ is a diagonal slice through 12,288 dimensions rather than a tidy axis, but the operation is identical — and this is the key point. The formula does not care about the number of dimensions. It works the same way for 2 numbers or 12,288. The dot product measures the overlap; the subtraction removes it.

This operation — subtracting the projection of a onto $\hat{r}$ — is sometimes written more compactly using a **projection matrix**:

$$a_{clean} = (I - \hat{r}\,\hat{r}^{\top})\,a$$

where I is the identity matrix ("leave everything alone") and $\hat{r}\,\hat{r}^{\top}$ is an outer product that extracts the refusal component. The expression $I - \hat{r}\,\hat{r}^{\top}$ is a fixed matrix that, applied to any activation, strips the refusal direction and leaves the rest. This compact form becomes important in the next section, because it is what gets baked into the weights.

How the model is actually structured: the residual stream

To understand where the edit lives, you need a picture of how information flows through a transformer layer. The key concept is the **residual stream**.

A transformer is not a chain of nodes and edges in the classical neural network sense. It is a stack of 88 identical-template blocks, each of which reads a shared vector, computes something, and *adds* its result back onto that vector. The shared vector — one per token position, always 12,288 numbers — is the residual stream. It is the model's working memory: the only place where layers communicate with each other.

Each layer's update is literally an addition:

```
residual_new = residual_old + attention_update + MLP_update
```

Layers never erase the stream; they only annotate it. By layer 88, the residual stream for the final token contains the accumulated contributions of every layer — and that final vector is what the model reads to decide which word comes next.

Inside each layer, there are seven large matrices doing work. Five of them — `q_proj`, `k_proj`, `v_proj` on the attention side and `gate_proj`, `up_proj` on the MLP side — are internal. They compute intermediate quantities but never deposit results into the residual stream directly. Two matrices are different:

- **`o_proj`** — the attention output projection. It takes the result of the attention computation (the weighted mixture of information gathered from other tokens) and performs a final matrix transformation before adding it to the residual stream.
- **`down_proj`** — the MLP down projection. The MLP first expands the token's 12,288-number state to 49,152 numbers (four times wider) using `up_proj` and `gate_proj`, does nonlinear computation there, then `down_proj` compresses the result back to 12,288 numbers and adds it to the residual stream.

These two — one per sub-block, two per layer, 176 across the whole model — are the only matrices whose outputs enter the shared highway. They are not passive conduits; they do real transformational work. But they are the *last* transformation before the

residual stream, which makes them the natural and complete intervention point.

Baking it in: editing the weights, not the activations

At this point you could apply the projection formula live, at every layer, for every token, via runtime hooks. This works and is useful for research — you can detach the hooks and the original model is back instantly. But it is not how you ship a model. The hook machinery must be present at every inference call, adding overhead and making the model file useless without the custom code.

The cleaner approach is to bake the removal into the model's weights directly, once, so the edited model is a standalone file that runs normally in any standard tool. The key insight is that instead of applying the projection matrix `(I − r̂ r̂ᵀ)` to every activation as it passes by, you can pre-apply it *to the write-head matrix itself*:

```
W' = (I − r̂ r̂ᵀ) W
```

which, when multiplied out, gives the form you see in code:

```
W' = W − r̂ (r̂ᵀ W)
```

Breaking this down: `r̂ᵀ W` computes, for each of `W`'s input features, "how much of this feature's output points in the refusal direction?" The result is a row of numbers — one per input feature. Multiplying back by `r̂` spreads those amounts across all 12,288 output dimensions according to the refusal direction's pattern, producing a matrix of the same shape as `W` that contains *only* the refusal-causing parts of `W`'s behaviour. Subtracting that produces `W'`: a matrix that behaves exactly like `W` on every input, except that its output is guaranteed to have zero component in the refusal direction.

The model still works because the edit removes **one direction** out of 12,288. The other ~12,287 directions pass through `W'` completely unchanged. Think of it as a filter that blocks one specific frequency while passing every other frequency without loss. Compare this to fine-tuning, which adjusts weights across the entire model in response to gradient signals — a process that touches everything and cannot easily be made this narrow.

Across 88 layers, each with both `o_proj` and `down_proj`, this means editing 176 matrices. Each edit is applied once, offline, before the model is deployed. After that, no special machinery is needed at inference time.

The tuning problem: strength and Heretic

The formula above is a full removal — `strength = 1.0`. In practice, refusal is not represented with equal intensity across all 88 layers, and the refusal direction slightly overlaps with adjacent useful behaviours like caution, hedging, or appropriate content warnings. A uniform full-strength removal across all layers is too blunt — it removes more than just refusal.

The edit is therefore parameterised with a per-layer strength dial:

```
W' = W − strength_layer × r̂ (r̂ᵀ W)
```

At `strength = 0`, no change at all. At `strength = 1`, full removal of the refusal direction. At `strength > 1`, the model is actively steered *away* from refusal — more aggressive, but increasingly likely to damage neighbouring behaviour. The trade-off is unavoidable:

- **Strength too low** — refusals survive. The layers where refusal is weakly represented get cleaned up, but the layers where it is strongly represented still generate it.
- **Strength too high** — you tear out useful behaviour that partially shared the direction. The model may become repetitive, lose coherent hedging, or show degraded reasoning in certain domains.
- **Strength varies per layer** — the right value for layer 15 may be very different from the right value for layer 55.

Finding the optimal strength across 176 matrices is a genuine multi-dimensional search problem — not something you can eyeball or solve analytically. This is where **Heretic** comes in.

Heretic is a framework built specifically to automate abliteration. It handles the direction computation, applies the per-layer edits, and then outsources the optimisation to **Optuna**, a Bayesian optimisation library. Rather than searching the parameter space exhaustively (which would require thousands of trials), Optuna builds a probabilistic model of which configurations are likely to be good based on results so far, and focuses new trials on promising regions. It is the same class of technique used in neural architecture search and hyperparameter tuning.

In our run, Heretic executed approximately 200 trials. Each trial applied a different strength configuration across layers, evaluated the result against both refusal rate and a general capability measure, and reported back. The search was guided by those two competing objectives simultaneously — which brings us to how the final configuration was chosen.

Measuring damage: KL divergence and the Pareto front

After applying the edit, you need to know two things: how much did refusals go down, and how much did everything else change?

Refusal rate is easy: run the held-out 100 harmful prompts and count. Ours dropped from 89/100 to 6/100.

General model damage is harder. The metric used is **KL divergence** (Kullback–Leibler divergence). Here is what it measures: at each token position, a language model produces a probability distribution over its entire vocabulary — something like "40% chance the next token is 'The', 15% chance it's 'A', 3% chance it's 'However'…" and so on across tens of thousands of tokens. KL divergence compares the original model's distribution to the edited model's distribution at the same position and computes a single number representing how different they are. A KL of zero means identical distributions; a high KL means the edit meaningfully changed what the model thinks is likely.

Critically, KL is measured on *neutral* prompts — everyday requests with no harmful content — where the refusal signal should not have been active in the first place. If the edit was surgical, these distributions should be nearly identical. If it was too aggressive,

they will diverge, telling you that the model's general text generation has been affected even when there was nothing to refuse.

These two metrics trade off, and that trade-off is the heart of the optimisation problem. More aggressive edits reduce refusal more but increase KL. Less aggressive edits preserve general capability but leave more residual refusals. The relationship between the two metrics, plotted across many trials, produces a **Pareto front** — a curve of configurations where you cannot improve one metric without worsening the other. Any configuration *on* the front is optimal in the sense that no better configuration exists for that trade-off point.

The job of the Heretic/Optuna search is to find the Pareto front efficiently and identify the most favourable point on it. In our project, configuration `r=3` was selected from the front — representing a strong removal (6/100 refusals) with a KL divergence of 0.119, indicating that general text generation was only minimally perturbed.

Why it fits in 69 MB

The final curiosity worth understanding is why the edit is small. Mistral Medium 3.5 128B in BF16 precision is approximately 256 GB. The LoRA adapter encoding the abliteration edit is 69 MB — roughly 1/3700th the size.

The reason is that the edit has low **rank**. Rank is a property of matrices that roughly measures "how many independent directions of information" a matrix contains. A full 12,288 × 12,288 matrix has rank up to 12,288 — it can potentially express any transformation in 12,288-dimensional space. But we only *changed* one direction. The delta between the original matrix `W` and the edited matrix `W'` is:

$$\Delta W = W' - W = -\hat{r}\,(\hat{r}^\top W)$$

This expression $\hat{r}\,(\hat{r}^\top W)$ is the **outer product** of one vector ($\hat{r}$, shape 12,288 × 1) with one row vector ($\hat{r}^\top W$, shape 1 × 12,288). An outer product of two vectors always has rank exactly 1 — regardless of how large the matrix is. A rank-1 matrix of size 12,288 × 12,288 can be fully represented by storing just two vectors: 12,288 + 12,288 = 24,576 numbers, instead of 150 million.

LoRA (Low-Rank Adaptation) is a standard framework for representing these compact weight changes. Instead of storing the full modified matrices, you store only the low-rank delta. The 69 MB adapter contains the rank-1 deltas for all 176 edited matrices. To deploy, you merge the adapter into the base model weights once, producing the full abliterated model. After merging, the result is a standard model file: no special infrastructure required.

The low rank is not an approximation or a compression shortcut — it is exact. The edit genuinely *is* rank-1 per matrix, because it was defined as a single-direction removal. This is one of the tidier mathematical properties of abliteration: the edit is simultaneously the most targeted possible change (one direction) and the most compact possible representation (rank-1).

The full pipeline in practice

It is worth tracing the actual sequence of steps from original model to deployed artifact, because the number of moving pieces is larger than the core method implies.

The model we started with — Mistral Medium 3.5 128B — does not fit on a single consumer GPU in full precision. Working with a model this size requires careful management of memory and compute at every step. The process ran as follows:

1. **Load the model in quantized form** for the activation collection pass. Running 800 forward passes (400 harmful + 400 harmless) on a 128B model in BF16 would require enormous GPU memory. Loading in 4-bit quantization (NF4) reduces the footprint enough to make the observation pass tractable, at a small cost to activation precision.

2. **Collect activations** at the target layers for each prompt. The last-token residual stream vector is captured after each target layer and written to disk.

3. **Compute the refusal direction** via difference of means. Normalise to get $\hat{r}$.

4. **Run the Heretic search**. For each Optuna trial, apply the parameterised edit to the write-head matrices, evaluate refusal rate and KL divergence, report back. Approximately 200 trials.

5. **Apply the best configuration** permanently to the weights and save the delta as a LoRA adapter (69 MB).

6. **Merge the adapter** into the base model weights to produce the standalone abliterated model.

7. **Quantize for deployment**. The merged BF16 model is quantized to GGUF Q4_K_M format for local inference, or kept in FP8 for server-side use. Quantization adds a small amount of additional numerical noise but reduces memory requirements by 4–8×.

8. **Evaluate**. Refusal rate on the held-out test set: 6/100. KL divergence on neutral prompts: 0.119. Qualitative probing across a range of topic areas to check for unintended capability degradation.

The operational complexity of this pipeline — loading a 128B model, managing quantization formats, running a multi-objective Bayesian search — is a significant part of why tools like Heretic exist. The core mathematical operation is a handful of lines of PyTorch. The infrastructure around it is where the real engineering effort goes.

What this tells us about the model

Perhaps the most interesting part of this whole exercise is not that we removed refusal — it is *that it was possible*. A behaviour that was distributed across 128 billion parameters during months of alignment training compressed down to something that could be characterised by running 400 prompts and captured in 24,576 numbers per matrix.

That says something real about how these models organise information. High-level semantic properties — including ones that are deliberately trained in — appear to cluster into linear directions in activation space, not tangled fractally through every weight. The training process, despite its complexity, seems to produce an internal geometry where concepts that are semantically related end up spatially adjacent, and concepts that are semantically opposite end up pointing in opposite directions. Refusal and harmlessness literally point away from each other in activation space — which is what makes their separation possible at all.

This has implications beyond abliteration. If high-level model behaviours are linearly organised, then in principle you could not just remove them but *measure* them, *steer* them, or *implant* them. Steering vectors — small additions to the residual stream that nudge the model toward a specific style or behaviour — work on the same principle. So does much of the current work in mechanistic interpretability: trying to reverse-engineer which directions in activation space correspond to which human-readable concepts.

A note on what this means practically

Abliteration is **dual-use**, and that deserves a direct acknowledgment rather than a footnote.

The obvious application in this project — reducing a model's tendency to refuse benign-but-edge-case requests, which aligned models sometimes over-refuse due to overly conservative training — is relatively innocuous. Many practical operator deployments have legitimate reasons to modify default refusal behaviour: medical platforms, security research tools, legal analysis systems. For these use cases, abliteration is a precise instrument for adjusting a model to match a specific operator context.

But the same technique can remove safety guardrails that were deliberately installed for good reasons. An abliterated model no longer has an internal refusal mechanism — not a bypassed one, a structurally absent one. That changes the risk profile in ways that matter. The model cannot be relied upon to self-govern harmful outputs; that responsibility moves entirely to the operator and the surrounding system.

The fact that this technique is accessible — it requires no GPU cluster, no training run, no dataset labelling — means it is within reach of anyone with sufficient hardware to load the model. That accessibility is part of why understanding the mechanism matters. The technique exists and is documented in public research. Pretending it does not is not a safety strategy.

Whether you find this reassuring or alarming probably depends on your role. What seems clear is that safety properties embedded in model weights are more fragile than they might appear — and that the field of mechanistic interpretability, which this technique is a practical application of, is going to have a lot more to say about this kind of thing in the coming years. Either way, it is a mechanism worth understanding properly.

This project used Mistral Medium 3.5 128B, the Heretic abliteration framework, Optuna for Bayesian search, and standard HuggingFace/transformers tooling. The full educational walkthrough is documented separately.